0787. Cheapest Flights Within K Stops

ategory: Advanced Graphs | LeetCode: https://leetcode.com/problems/cheapest-

Problem

- Directed flights (from, to, price). Find the cheapest price from src to dst using at most k stops (k + 1 edges), or -1.

Intuition

- Bellman-Ford limited to k + 1 rounds: after round i, prices[] holds the cheapest cost using at most i edges.
- Each round must relax from a snapshot of the previous round, or a single round could chain several edges.

Brute Force

- DFS over all paths up to k + 1 edges: exponential.

Optimal Approach

- Prices = infinity, prices[src] = 0.
- Repeat k + 1 times: tmp = copy(prices); for each flight (u, v, w) with prices[u] finite: tmp[v] = min(tmp[v], prices[u] + w); prices = tmp.
- Return prices[dst] or -1.

Worked Example

```
n = 4, flights [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src 0, dst 3, k 1.

round 1 (1 edge):  1=100
round 2 (2 edges): 2=200, 3=700
answer 700 (0->2->3 = 400 would need 2 stops)
```

Java Solution

```java
public class CheapestFlightsWithinKStops {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        int[] prices = new int[n];
        Arrays.fill(prices, Integer.MAX_VALUE);
        prices[src] = 0;
        for (int round = 0; round <= k; round++) {
            int[] tmp = prices.clone();
            for (int[] f : flights) {
                if (prices[f[0]] == Integer.MAX_VALUE) continue;
                tmp[f[1]] = Math.min(tmp[f[1]], prices[f[0]] + f[2]);
            }
            prices = tmp;
        }
        return prices[dst] == Integer.MAX_VALUE ? -1 : prices[dst];
    }
}
```

Complexity

- Time: O(k E). Space: O(n).

Pitfalls

- Relaxing in place without the snapshot gives wrong (too cheap) answers.